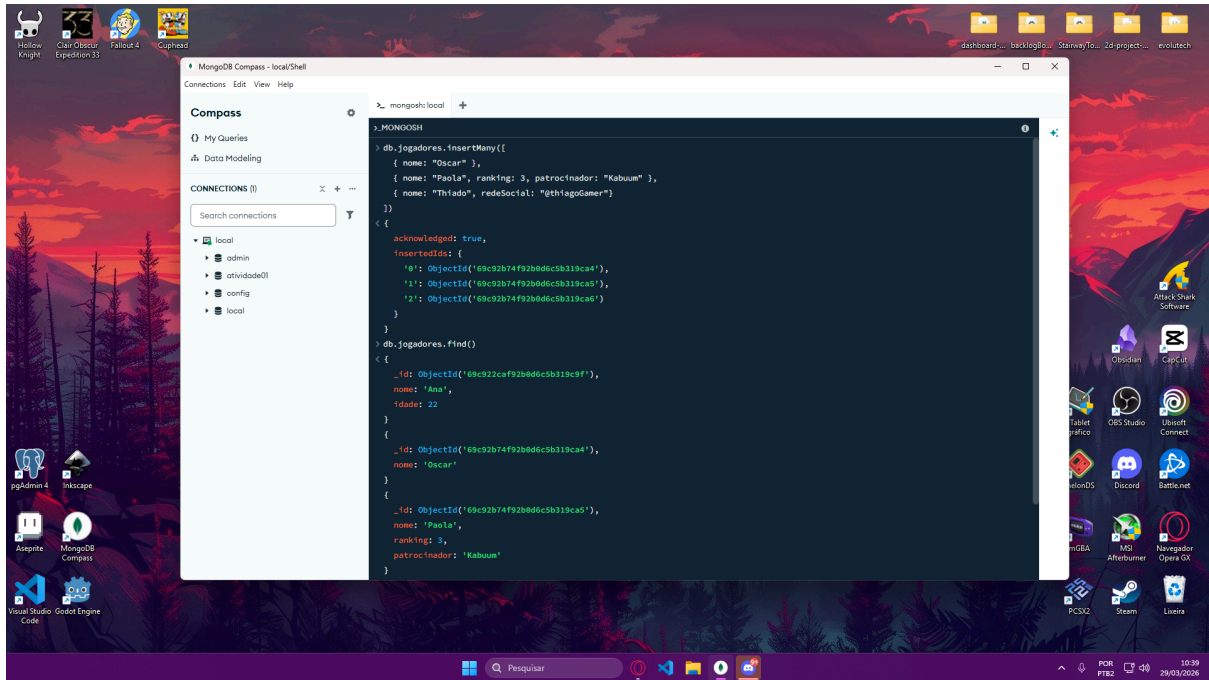


Atividade Aula 03

Parte 01: Banco sem Estrutura Definida com Schema

Tarefa 1: Inserção livre



Tarefa 2: Análise crítica

1- Todos os documentos possuem o mesmo padrão?

Não. Os documentos não possuem o mesmo padrão, porque o banco de dados não exige um schema padrão. Cada documento pode ter campos diferentes.

2- Isso pode gerar problemas em consultas futuras?

Sim. Como os documentos não possuem os mesmos campos ao realizar consultas esperando determinado campo pode não retornar nada ou dados inconsistentes.

3- Cite um risco desse modelo em sistemas grandes.

O risco é justamente o retorno dos dados inconsistentes e dependendo de como a aplicação consome esses dados o uso do sistema pode ter erros ocasionados pela falta de campos em determinadas consultas.

Parte 02: Banco com Estrutura definida com Schema

Tarefa 3: Inserção correta

```
>_MONGOSH  
  
> db.jogadores.insertOne({  
  nome: "Marcio",  
  nick: "th3ch0s3n0ne_",  
  ranking: 100  
})  
< {  
  acknowledged: true,  
  insertedId: ObjectId('69c92dadf92b0d6c5b319ca7')  
}
```

Tarefa 4: Inserção inválida

```
>_MONGOSH  
  
> db.jogadores.insertOne({  
  nome: "Ricardo"  
})  
❌ > MongoServerError: Document failed validation  
  
> db.jogadores.insertOne({  
  nome: "Oliver",  
  nick: "Oli",  
  ranking: "120"  
})  
❌ > MongoServerError: Document failed validation  
BD_CE > |
```

Tarefa 5: Comparação técnica

1- Qual banco oferece maior segurança para a produção?

O banco que possui estrutura definida com Schema é mais seguro. Como ele possui validação dos documentos, só é possível inserir documentos com o mesmo padrão e campos. Isso garante que não haverá inconsistência entre documentos e seus dados, diminuindo erros em consultas.

2- Qual banco oferece flexibilidade para prototipação?

O banco sem estrutura definida, pois permite inserção de documentos com diferentes campos e estruturas, justamente por não exigir uma estrutura padrão.

3- Em qual cenário você utilizaria cada modelo?

Para um sistema em produção e já em uso o BD_CE é o ideal, por garantir a consistência dos dados e pela segurança da validação presente no Schema, evitando erros em consultas e consequentemente melhor desempenho para o sistema.

Em um cenário de desenvolvimento em etapas iniciais, o BD_SE permite um desenvolvimento sem grandes restrições e alterações em tempo de desenvolvimento/pré-deploy.

Parte 03: Raciocínio de Arquitetura

Para um sistema que será escalado para milhares de usuários simultâneos, integrado com uma API pública, recomendo o BD_CE.

Este modelo de banco garante maior segurança dos dados e maior consistências nas consultas ao banco pela aplicação, mitigando erros e falhas quando o sistema consumir a API e puxar os dados do Banco. Fora isso vai restringir os erros ao cadastrar novos usuários e manter consistência entre os dados.